

Serialisierung und Datei-I/O – Teil 2

Textdateien schreiben und lesen

Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

Prof. Dr. Martin Burger { 29. Januar 2026 }

inno



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstruktoren und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan 
überlebt den
ersten
Kontakt  mit
der Realität .

Frei nach Helmuth Karl
Bernhard von Moltke

Serialisierung und Datei-I/O

 Plan für diese Woche

 Zustand speichern

 Serialisierung

 Interface Serializable

 Flüchtige Instanzvariablen

 Deserialisierung

 Versions-ID

 Textdatei schreiben

 Klasse File

 Puffer

 Textdatei lesen

 Try-With-Resources

 Pair Programming

 Spaß

📖 Serialisierung und Datei-I/O

🔗 Verbindungen herstellen

👤 Tauschen Sie sich mit einer Nachbar:in aus!

✍️ Wie schreibt man eine Textdatei in Java?

🗄️ Was repräsentiert die Klasse `File`?

🔥 Wie hängen Puffer und Streams zusammen?

📖 Wie liest man eine Textdatei?

👉 Wie hängen Try-With-Resources und der `finally`-Block zusammen?

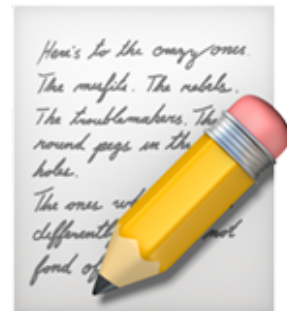
👤 Welche Fragen haben Sie zu diesem Kapitel?

16 Serialisierung und Datei-I/O

Objekte (und Text) speichern



Objekte lassen sich flach drücken und wieder aufblasen. Objekte besitzen Zustand und Verhalten. Das *Verhalten* lebt in der *Klasse*, während sich der *Zustand* in den einzelnen Objekten befindet. Was passiert aber, wenn es Zeit ist, den Zustand eines Objekts zu speichern? Wenn Sie ein Spiel schreiben, benötigen Sie eine Möglichkeit, den Spielstand zu sichern und wiederherzustellen. Wenn Sie eine App für die Erstellung von Diagrammen programmieren, müssen Sie die Datei irgendwie speichern und wieder öffnen können. Soll Ihr Programm Zustände speichern, können Sie das entweder *auf die harte Tour machen*, indem Sie jedes Objekt befragen und dann mühsam die Werte aller Instanzvariablen in eine Datei schreiben, die ein von Ihnen entworfenes Format hat. Oder **Sie gehen einfach nach OO-Art vor**. Hierfür wird das Objekt selbst gefriergetrocknet/flach gedrückt/persistent gemacht/dehydriert. Um es zurückzubekommen, wird es wieder aufgetaut/aufgeblasen/wiederhergestellt/rehydriert. Trotz dieser Möglichkeiten kann es *manchmal* sein, dass es nur auf die harte Tour geht, besonders wenn die Datei, die Ihre App speichern muss, von einer Nicht-Java-Applikation gelesen werden muss. Daher sehen wir uns in diesem Kapitel beide Wege an. Und weil alle I/O-Operationen (I/O = E/A, Ein-/Ausgabe) potenziell gefährlich sind, werfen wir auch gleich einen Blick auf bessere Möglichkeiten des Exception-Handlings.



Textdatei schreiben

```
1 import java.io.FileWriter;
2 import java.io.IOException;
3
4 class FileWriterDemo {
5     public static void main(String[] args) {
6         final String fileName = "/tmp/output.txt";
7
8         // Since we want to write text, not bytes, we use FileWriter instead of FileOutputStream:
9         // "FileWriter is meant for writing streams of characters. For writing streams of raw bytes,
10        // consider using a FileOutputStream."
11        FileWriter writer = null;
12
13        try {
14            writer = new FileWriter(fileName);
15            writer.write("Hello World from Java!");
16        } catch (IOException e) {
17            // In real code, our response to this exception would be some sort of recovery, for example.
18            System.err.format("I/O error writing to file: %s%n", fileName);
19        } finally {
20            if (writer != null) {
21                try {
22                    writer.close();
23                } catch (IOException e) {
24                    // In real code... you get the idea.
25                    System.err.format("I/O error closing file: %s%n", fileName);
26                }
27            }
28        }
29    }
30 }
31
```

STDIN

Input for the program (Optional)

Output:

Program did not output anything!

! Quiz



Textdatei schreiben



Ich mache einige Aussagen –
sind diese Aussagen richtig
oder falsch?



Bei **richtigen** Aussagen **stehen**
Sie **kurz auf!**



Bei **falschen** Aussagen **bleiben**
Sie **sitzen!**



Um eine Textdatei zu schreiben, beginnen Sie mit einem `FileWriter`-Anschluss-Stream.

Richtig!

Um einen `String` in die Textdatei zu schreiben,
verwenden Sie die Methode
`FileWriter.writeObject("Some String")`.

Die Methode `FileWriter.write("Some String")`.

Ist die Datei, in die der `FileWriter` schreiben soll, noch nicht vorhanden, so wird sie durch den `FileWriter` erzeugt.

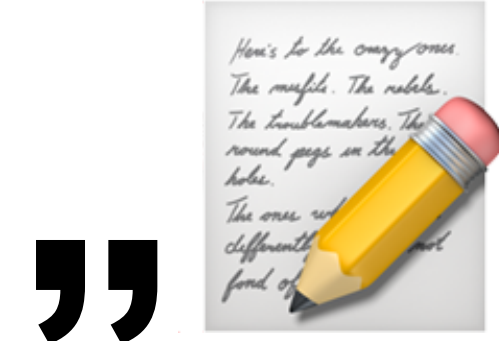
Richtig!

Sowohl die Serialisierung als auch die Speicherung in einer Textdatei können Ausnahmen auslösen.

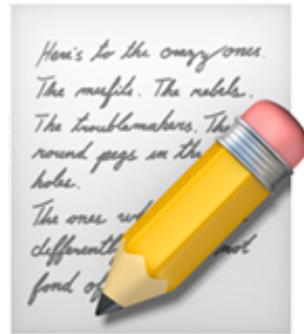
Richtig!



Welcher **eine** Gedanke zu



„Textdatei schreiben“ war
für Sie am wichtigsten?



Schreiben Sie ihn auf!



Durchatmen und



Sortieren



Klasse File

Jetzt Seite 564 wiederholen!



Sie haben dafür 3 Minuten Zeit.

! ? Quiz

🗄️ Klasse File

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Ein `File`-Objekt kann eine Datei repräsentieren, die sich unter einem bestimmten Dateipfad befindet. Es repräsentiert nicht den Inhalt der Datei.

Richtig!

File-Objekte repräsentieren Dateien, aber keine Verzeichnisse.

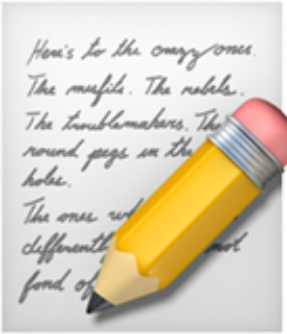
Nicht nur Dateien.

Sie können Verzeichnisse durchsuchen,
erstellen und löschen, indem Sie ein File-
Objekt verwenden.

Richtig!

Die meisten Streams, die Dateinamen als `String` annehmen, können auch mit `File`-Objekten arbeiten, deren Verwendung in der Regel noch sicherer ist.

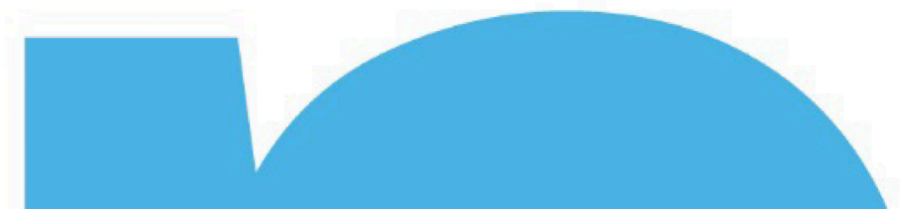
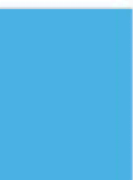
Richtig!

 Welcher **eine** Gedanke zu
„ Klasse File“ war für Sie
am wichtigsten?  Schreiben
Sie ihn auf!

 Durchatmen und  Sortieren



Puffer



BufferDemo.java

+

Puffer

AI

NEW

JAVA

RUN

1

import java.io.BufferedWriter;

2

import java.io.File;

3

import java.io.FileWriter;

4

import java.io.IOException;

5

6

class BufferDemo {

7

public static void main(String[] args) {

8

final String fileName = "/tmp/output.txt";

9

10

// We do not want to accidentally overwrite the file, so we will check to see if the file

11

// exists.

12

final File file = new File(fileName);

13

if (file.exists()) {

14

System.err.format("File %s already exists!\n", fileName);

15

return;

16

}

17

18

// We use BufferedWriter because it is more efficient than FileWriter:

19

// "Writes text to a character-output stream, buffering characters so as to provide for the

20

// efficient writing of single characters, arrays, and strings."

21

BufferedWriter writer = null;

22

23

try {

24

// FileWriter is a connection stream, which in our case means it connects to a file.

25

// BufferedWriter is a chain stream, which means it sits on top of another stream (in this

26

// case, a FileWriter).

27

writer = new BufferedWriter(new FileWriter(file));

28

29

writer.write("Hello World from Java - in a more efficient way!");

30

31

// We could use writer.flush() here, but that's not necessary: writer.close() will do it

32

// automatically.

33

// writer.flush();

34

} catch (IOException e) {

35

// In real code... you already figured that out.

36

System.err.format("I/O error writing to file: %s\n", file);

37

} finally {

38

if (writer != null) {

39

try {

40

// This will also flush the buffered stream and close the underlying file stream.

41

writer.close();

42

} catch (IOException e) {

43

// In real code...

44

System.err.format("I/O error closing buffered stream on file: %s\n", file);

45

}

46

}

47

}

48

}

49

}

50

STDIN

Input for the program (Optional)

Output:

Program did not output anything!

! ? Quiz

🐡 Puffer

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Der `FileWriter` kann mit einem
`BufferedWriter` gekoppelt werden.
Dadurch wird das Schreiben effizienter.

Richtig!

Erst wenn der Puffer des `BufferedWriter` voll ist, werden die darin enthaltenen Daten vom `FileWriter` auf die Festplatte geschrieben.

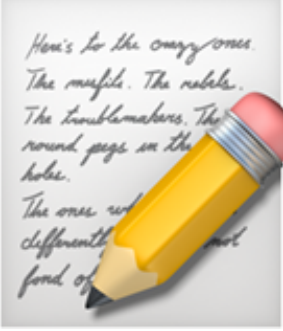
Richtig!

Wenn Sie den `BufferedWriter` schließen,
schließt er auch alle darunter liegenden
Streams.

Richtig!

Es ist nicht möglich, einen Puffer zum Senden von Daten zu zwingen, bevor dieser vollständig gefüllt ist.

Doch, sie können einen Puffer „flushen“.

 Welcher **eine** Gedanke zu
„ Puffer“ war für Sie am
wichtigsten?  Schreiben Sie
ihn auf!

 Durchatmen und  Sortieren



Textdatei lesen


```
1 import java.io.BufferedReader;
2 import java.io.File;
3 import java.io.FileReader;
4 import java.io.IOException;
5
6 class FileReaderDemo {
7     public static void main(String[] args) {
8         final String fileName = "input.txt";
9
10        // We check for the existence of the file before attempting to read it.
11        final File file = new File(fileName);
12        if (!file.exists()) {
13            System.err.format("File %s does not exist!\n", fileName);
14            return;
15        }
16
17        // Since we are going to read from a file, we need a Reader.
18        BufferedReader reader = null;
19
20        try {
21            // BufferedReader provides buffering of data for fast reading.
22            // FileReader is intended for reading streams of characters.
23            reader = new BufferedReader(new FileReader(file));
24
25            // The single line read from the underlying file.
26            String line;
27            // As long as there is a line available: print that line.
28            while ((line = reader.readLine()) != null) {
29                System.out.println(line);
30            }
31        } catch (IOException e) {
32            // In real code... you already got the idea.
33            System.err.format("Error reading from file: %s\n", file);
34        } finally {
35            if (reader != null) {
36                try {
37                    reader.close();
38                } catch (IOException e) {
39                    // In real code...
40                    System.err.format("Error closing buffered stream on file: %s\n", file);
41                }
42            }
43        }
44    }
45 }
46
```

STDIN

Input for the program (Optional)

Output:

Click on RUN button to see the output

! ? Quiz

📖 Textdatei lesen

🤔 Ich mache einige Aussagen –
sind diese Aussagen richtig
oder falsch?

👍 Bei **richtigen** Aussagen **stehen**
Sie **kurz auf!**

👎 Bei **falschen** Aussagen **bleiben**
Sie **sitzen!**



Um eine Textdatei zu lesen, beginnen Sie mit einem `FileReader`-Anschluss-Stream.

Richtig!

Eine gängige Art, Daten zu lesen, sieht wie folgt aus:
Einlesen in einer while-Schleife (im Test der Schleife) und
Abbruch, wenn es nichts mehr zu lesen gibt (das Ergebnis
der Lesemethode ist `null`).

Richtig!

Der `FileReader` kann mit einem `BufferedReader` verkettet werden. Dies ermöglicht ein effizienteres Lesen.

Richtig!

Sowohl `FileReader` als auch `FileWriter`
können optional gepuffert werden.

Richtig!

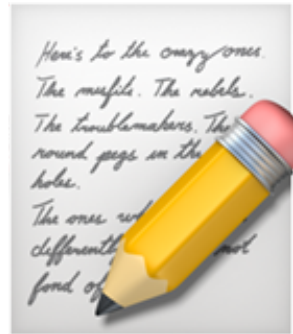


Welcher **eine** Gedanke zu



„Textdatei lesen“ war für

Sie am wichtigsten?



Schreiben Sie ihn auf!



Durchatmen und



Sortieren

👉 **Try-With-Resources**

**Jetzt Seiten 574
bis 577
wiederholen!**



Sie haben dafür 9 Minuten Zeit.


```
1 import java.io.BufferedReader;
2 import java.io.File;
3 import java.io.FileReader;
4 import java.io.IOException;
5
6 class TryWithResourcesDemo {
7     public static void main(String[] args) {
8         final File file = new File("input.txt");
9
10        // We use a try-with-resources block to automatically close the reader.
11        try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
12            // The line read one by one from the underlying file.
13            String line;
14            // As long as there is a line of text available: print that line of text.
15            while ((line = reader.readLine()) != null) {
16                System.out.println(line);
17            }
18        } catch (IOException e) {
19            // In real code..
20            System.err.format("I/O error reading from file: %s\n", file);
21        }
22    }
23 }
24
```

STDIN

Input for the program (Optional)

Output:

Click on RUN button to see the output

! ? Quiz

👉 Try-With-Resources

🤔 Ich mache einige Aussagen –
sind diese Aussagen richtig
oder falsch?

👍 Bei **richtigen** Aussagen **stehen**
Sie **kurz auf!**

👎 Bei **falschen** Aussagen **bleiben**
Sie **sitzen!**



Wenn Sie try-with-resources verwenden, erzeugt der Compiler einen `finally`-Block. Sie können ihn nicht im Quellcode sehen, aber er ist im Bytecode vorhanden.

Richtig!

In einer try-with-resources- (TWR-) Anweisung darf nur eine I/O-Ressource deklariert und verwendet werden.

Falsch, es können mehrere behandelt werden.

Bei der Verwendung von try-with-resources-
(TWR-) Anweisungen sind keine expliziten
finally-Blöcke zulässig.

Doch! Dieser finally-Block wird nach dem Schließen der Ressourcen ausgeführt.

Nur Klassen, die `AutoCloseable` implementieren, können in try-with-resources-(TWR-) Anweisungen verwendet werden.

Richtig!

 Welcher **eine** Gedanke zu
„ Try-With-Resources“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

Pair Programming

🎓 Textdateien schreiben

🤝 Pair Programming

- 🎯 Erstellen Sie eine Anwendung, die Eingaben in Form von Artikeln für eine Einkaufsliste entgegennimmt und in einer Textdatei speichert.
- ✅ Die Anwendung soll die Benutzer:in auffordern, Artikel für die Einkaufsliste einzugeben.
- ✅ Nach jedem Artikel soll die Benutzer:in die Möglichkeit haben, einen weiteren Artikel hinzuzufügen oder den Prozess zu beenden.
- ✅ Verwenden Sie `FileWriter` und `BufferedWriter`, um die Textdatei zu erstellen.



Textdateien schreiben

Pair Programming

 **Erstellen Sie eine Anwendung, die Eingaben in Form von Artikeln für eine Einkaufsliste entgegennimmt und in einer Textdatei speichert.**

- ✓ Die Anwendung soll die Benutzer:in auffordern, Artikel für die Einkaufsliste einzugeben.
- ✓ Nach jedem Artikel soll die Benutzer:in die Möglichkeit haben, einen weiteren Artikel hinzuzufügen oder den Prozess zu beenden.
- ✓ Verwenden Sie `FileWriter` und `BufferedWriter`, um die Textdatei zu erstellen.

- ✓ Arbeiten Sie mit einem `File`-Objekt, um Ihre Anwendung abzusichern.
- ✓ Verwenden Sie `Try-With-Resources`, um sicherzustellen, dass die Ressourcen korrekt geschlossen werden.
- ✓ Jeder Artikel soll in eine neue Zeile der Textdatei geschrieben werden.
- 🏁 Am Ende sollten Sie ein Programm haben, das Benutzereingaben zeilenweise in eine Textdatei schreibt und gegenüber Fehlern robust ist.

EinkaufslistenApp.java

+

Einkaufsliste

AI

NEW

JAVA

RUN

```
1 import java.io.BufferedWriter;
2 import java.io.File;
3 import java.io.FileWriter;
4 import java.io.IOException;
5 import java.util.Scanner;
6
7 public class EinkaufslistenApp {
8     public static void main(String[] args) {
9         final String listFileName = "/tmp/shoppinglist.txt";
10
11         final File listFile = new File(listFileName);
12         if (listFile.exists()) {
13             System.err.format("File %s already exists!\n", listFile);
14             return;
15         }
16
17         try (Scanner scanner = new Scanner(System.in);
18              BufferedWriter writer = new BufferedWriter(new FileWriter(listFile))) {
19
20             System.out.println("Enter a shopping item (or '.' to exit):");
21
22             while (true) {
23                 System.out.print("> ");
24                 String input = scanner.nextLine();
25                 // Echo input and force new line in OneCompiler.
26                 System.out.println(input);
27
28                 if (".".equals(input)) {
29                     // Exit loop, thus ending the program.
30                     break;
31                 }
32
33                 if (input.trim().isEmpty()) {
34                     // Skip empty lines (i.e., lines that contain only whitespace), but continue the loop.
35                     continue;
36                 }
37
38                 writer.write(input);
39
40                 // Write system-specific line delimiters: \n on Unix, \r\n on Windows.
41                 writer.newLine();
42             }
43
44             System.out.format("Shopping list written to file: %s\n", listFile);
45         } catch (IOException e) {
46             System.err.format("Error writing to file: %s\n", listFile);
47             System.err.format("Error message: %s\n", e.getMessage());
48         }
49
50         System.out.println("Contents of the shopping list:");
51         // Scanner can be used not only to read from the console, but also from files.
52         try (Scanner fileScanner = new Scanner(listFile)) {
53             while (fileScanner.hasNextLine()) {
54                 System.out.println(fileScanner.nextLine());
55             }
56         } catch (IOException e) {
57             System.err.format("Error reading from file: %s\n", listFile);
58             System.err.format("Error message: %s\n", e.getMessage());
59         }
60     }
61 }
62
```

STDIN

Milk

Eggs

.

Output:

Enter a shopping item (or '.' to exit):

> Milk

>

> Eggs

> .

Shopping list written to file: /tmp/shoppinglist.txt

Contents of the shopping list:

Milk

Eggs

 Welcher **eine** Gedanke zu
„ Textdateien schreiben“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

 **Zielgerade**

↔ Teach Back

📖 Serialisierung und Datei-I/O

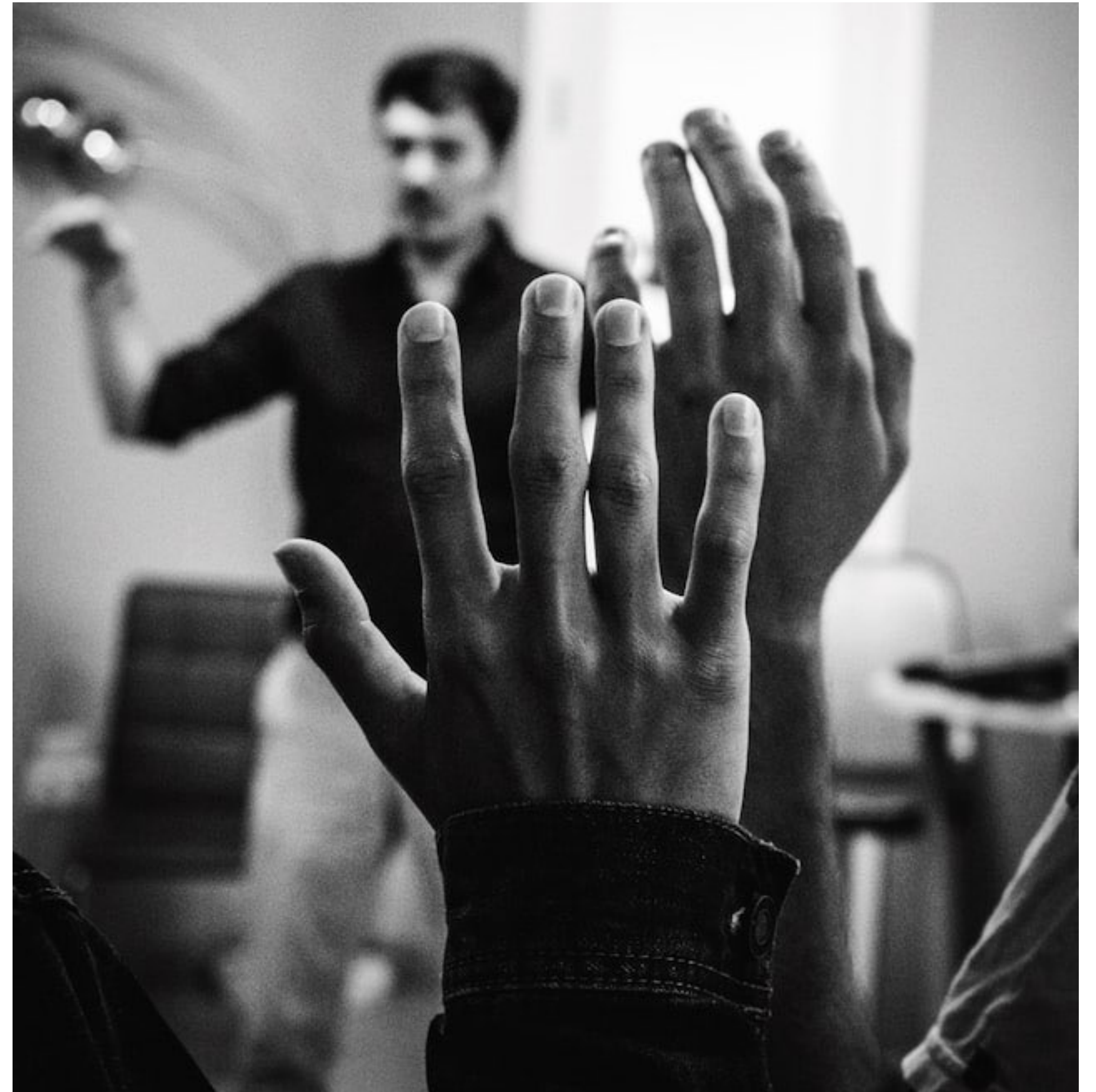
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Serialisierung und Datei-I/O?**
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Serialisierung und Datei-I/O



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus

